

Cahier des charges

Cahier des charges - BagTrip

1. Contexte et objectif

BagTrip est une application de planification de voyages composee de trois produits relies :

- une application mobile Flutter pour les voyageurs.
- une API FastAPI qui centralise les données, les règles metier et les integrations externes.
- un panel d'administration Next.js pour superviser la plateforme.

L'objectif du projet est de permettre a un utilisateur de créer, organiser, suivre et partager un voyage de bout en bout, avec assistance IA pour la planification, la gestion du budget, le suivi des activités, les notifications et la reservation de services de voyage.

2. Périmètre fonctionnel

Le produit couvre le cycle complet d'un voyage :

- création du compte et authentification.
 - personnalisation du profil voyageur.
 - creation du voyage en mode manuel ou assisté par IA.
 - gestion des activités, hébergements, vols, bagages et budget.
 - partage du voyage avec d'autres utilisateurs.
 - suivi du voyage en cours avec notifications et mode in-trip.
 - retour post-voyage avec feedback et souvenirs.
 - supervision admin des données et des utilisateurs.
-

3. Parties prenantes et utilisateurs

3.1 Utilisateur final

L'utilisateur final est un voyageur qui souhaite planifier un trip seul ou a plusieurs, avec ou sans assistance IA.

3.2 Contributeur partage

Un voyage peut être consulté ou enrichi par des viewers invites, selon les droits accordes sur le trip.

3.3 Administrateur

L'administrateur gère les données de la plateforme via le panel admin : utilisateurs, voyages, réservations, notifications, feedbacks et indicateurs.

4. Cas d'usage principaux

4.1 Authentification et compte

Le système doit permettre :

- l'inscription par email et mot de passe.
- la connexion classique.
- la connexion via Google.
- la connexion via Apple.
- la rotation des refresh tokens.
- la déconnexion.
- la consultation et la mise à jour du profil courant.

4.2 Onboarding et personnalisation

L'utilisateur doit pouvoir compléter son profil voyageur afin de renseigner ses préférences de voyage, son style, son budget et ses compagnons habituels.

4.3 Création de voyage

Le système doit proposer deux modes de création :

- un mode manuel où l'utilisateur choisit lui-même sa destination.
- un mode IA où le système propose des destinations puis génère un itinéraire complet.

Le wizard de création doit couvrir les dates, le budget, le nombre de voyageurs, la destination, puis la revue finale avant validation.

4.4 Planification IA

L'IA doit pouvoir produire un plan de voyage composé de destinations, activités, hébergements, bagages et estimation budgétaire, en s'appuyant sur des données externes réelles lorsque disponibles.

4.5 Gestion du voyage

L'utilisateur doit pouvoir :

- consulter le détail du voyage.
- suivre les activités jour par jour.

- gérer les hébergements.
- gérer les bagages.
- suivre le budget.
- accéder a un mode in-trip avec timeline et indicateur de l'heure courante.

4.6 Vols et transports

Le produit doit intégrer la recherche de vols, les informations de transport et, lorsque pertinent, la réservation ou la préparation de booking intents.

4.7 Partage

Le système doit permettre d'inviter d'autres utilisateurs sur un trip, de gérer leurs permissions et de révoquer l'accès si besoin.

4.8 Notifications

Le système doit envoyer des notifications push et locales pour les étapes importantes du voyage : départ, vol imminent, activités, resume quotidien, budget et fin de voyage.

4.9 Post-voyage

Le système doit proposer un retour post-voyage avec feedback et transition vers la cloture du trip.

4.10 Administration

Le panel admin doit permettre de superviser les utilisateurs, les voyages, les reservations, les activites, les hebergements, les bagages, les budgets, les partages, les feedbacks et les notifications.

5. Exigences fonctionnelles par module

5.1 Authentification

- inscription email/mot de passe.
- login email/mot de passe.
- OAuth Google.
- OAuth Apple.
- récupération du profil courant via /v1/auth/me.
- refresh automatique des tokens.
- déconnexion et invalidation du refresh token.
- gestion du statut d'authentification pour mobile et admin panel.

5.2 Profil voyageur

- modification des informations de base.
 - stockage des préférences de voyage.
 - prise en compte du niveau de completion du profil.
 - liaison avec les recommandations et la planification IA.
-

5.3 Creation de voyage

- saisie des dates en mode exact, mois ou flexible.
 - choix du nombre de voyageurs.
 - choix ou estimation du budget.
 - recherche manuelle de destinations.
 - suggestions IA de destinations.
 - generation du plan de voyage.
 - validation finale et creation du trip.
-

5.4 Trip detail

- affichage du resume du voyage.
 - sections activités, vols, hébergement, bagages et budget.
 - état du voyage et progression.
 - actions de modification ou consultation selon le role.
-

5.5 Activités

- CRUD des activités.
 - gestion des horaires.
 - suggestions IA.
 - affichage en mode voyage.
 - liaison avec les notifications.
-

5.6 Hébergements

- consultation et gestion des hébergements.
- integration des prix et informations de recherche.

- usage dans la generation IA et le budget.
-

5.7 Bagages

- gestion de checklist bagages.
 - suggestions IA d'articles essentiels.
 - suivi des items emballés.
 - rappel de preparation avant depart.
-

5.8 Budget

- estimation initiale et détaillée.
 - suivi des dépenses.
 - repartition par catégorie.
 - alertes de dépassement.
 - prise en compte des estimations IA.
-

5.9 Sharing

- invitation de participants.
 - gestion des rôles et permissions.
 - consultation partagée du voyage.
 - retrait des accès.
-

5.10 Notifications

- notifications serveur via FCM.
 - notifications locales planifiées dans l'application.
 - centre de notifications avec pagination et lecture/non-lecture.
 - deep links vers les écrans concernés.
-

5.11 Mode in-trip

- detection automatique du changement d'etat du voyage.
- affichage des activités du jour.
- indicateur temporel courant.

- vues adaptées aux états planned, ongoing et completed.
-

5.12 Admin panel

- tableau de bord avec KPIs.
 - gestion des utilisateurs et de leurs plans.
 - gestion des trip data.
 - consultation des reservations et intents.
 - suivi des feedbacks.
 - envoi de notifications admin.
 - visualisation des métriques et des données metier.
-

6. Exigences techniques

6.1 Architecture globale

Le systeme est organisé en monorepo avec trois applications :

- bagtrip/ pour le mobile Flutter.
- api/ pour le backend FastAPI.
- admin-panel/ pour le front admin Next.js.

Le tout est orchestre par Makefile et compose.yml pour le dev local.

6.2 Mobile

- architecture BLoC + Repository + Result.
 - injection de dependances via GetIt.
 - navigation type-safe avec go_router.
 - cache local Hive avec TTL.
 - internationalisation FR/EN.
 - accès offline partiel.
 - tests unitaires et tests d'integration Flutter.
-

6.3 API

- FastAPI sous Uvicorn.
- PostgreSQL 15.

- SQLAlchemy + Alembic.
 - gestion des erreurs type.
 - middleware de rate limiting.
 - integrations externes Amadeus, Stripe, Firebase, Unsplash, AirLabs et Open-Meteo.
 - pipeline IA LangGraph et SSE streaming.
-

6.4 Admin panel

- Next.js 15 avec App Router.
 - TypeScript.
 - React Query pour l'état serveur.
 - Radix UI, Tailwind CSS et composants partages.
 - Cypress pour les tests E2E.
-

7. Contraintes non fonctionnelles

7.1 Performance

- temps de réponse compatible mobile.
 - chargement progressif des données.
 - cache local pour les données récupérées fréquemment.
 - traitement asynchrone des notifications et de la planification IA.
-

7.2 Disponibilite et resilience

- tolerance aux pannes des services externes.
 - fallback lorsque certaines integrations sont absentes.
 - gestion propre des refresh tokens et des sessions.
 - reprise partielle hors ligne cote mobile.
-

7.3 Sécurité

- JWT avec refresh token rotation.
- cookies httpOnly pour le panel admin.
- OAuth Google et Apple.
- protection des routes admin.

- rate limiting sur les endpoints sensibles.
 - séparation des secrets via variables d'environnement.
-

7.4 Qualité et maintenabilité

- linting automatisé.
 - formatting obligatoire.
 - tests frontend et backend.
 - CI/CD avec quality gates.
 - documentation fonctionnelle et technique maintenue.
-

7.5 Accessibilité et UX

- support de l'accessibilité mobile.
 - design system cohérent.
 - adaptation iOS / Android.
 - support FR/EN.
 - mode sombre.
-

8. Integrations externes

Le projet dépend des services tiers suivants :

- Amadeus pour les vols, les hôtels et certaines recherches de destinations.
 - Stripe pour les paiements et abonnements.
 - Firebase Admin / FCM pour les notifications push.
 - Open-Meteo pour la meteo.
 - Unsplash pour les images de couverture.
 - AirLabs pour certaines informations de vol.
 - un fournisseur LLM compatible OpenAI pour l'agent IA.
-

9. Règles de gestion

- les plans utilisateur sont FREE, PREMIUM et ADMIN.
- les quotas IA varient selon le plan.
- certaines fonctionnalités sont réservées aux utilisateurs premium ou admin.

- les trips peuvent être visibles selon les droits owner/viewer.
 - les notifications et les suggestions doivent être cohérentes avec l'état du voyage.
-

10. Livrables attendus

- application mobile Flutter opérationnelle.
 - API backend documentée et exécutable localement.
 - panel d'administration fonctionnel.
 - pipeline CI/CD avec tests et quality gates.
 - documentation projet à jour.
 - configuration Docker pour le dev local.
 - jeux de données ou seeds de démarrage.
-

11. Hors périmètre ou points à préciser

Certains sujets apparaissent comme incomplets ou à finaliser dans la documentation actuelle :

- reset de mot de passe. (implémenté)
 - suppression de compte. (implémenté)
 - vérification email.
 - persistance du brouillon de plan de voyage.
 - couverture de tests plus large sur certains flux backend.
-

12. Critères d'acceptation

Le projet peut être considéré conforme si :

- un utilisateur peut s'inscrire, se connecter et compléter son profil.
 - un voyage peut être créé en mode manuel ou IA.
 - les activités, bagages, budget et hébergements sont visibles et cohérents.
 - les notifications principales sont reçues au bon moment.
 - le partage de voyage fonctionne.
 - le panel admin permet la supervision des données.
 - l'ensemble tourne en local via les commandes de dev prévues.
 - la CI valide les tests et le lint.
-

13. Conclusion

BagTrip est une plateforme de voyage très complète, orientée planification assistée, suivi contextuel et expérience mobile continue. Le cahier des charges ci-dessus formalise le périmètre fonctionnel et technique de la solution à partir de la documentation existante, tout en signalant les zones encore à consolider.